# Project APEX Production Debugging Casebook

*Fifteen failures investigated from symptom to regression test*

OpenAI

August 2026

## Table of Contents

## 2.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 2.3 High-information first probe

Print one raw value, source unit, canonical value and expected hand conversion at the adapter boundary.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 2.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 2.5 Root cause

Conversion is performed both by the source adapter and a downstream feature transform.

## 2.6 Correct repair

Choose one canonical unit, convert once in the adapter, store unit metadata, and delete downstream implicit conversion.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 2.7 Regression test

A fixture containing 360 km/h must become exactly 100 m/s and remain 100 m/s through window construction.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 2.8 The tempting wrong repair

Changing model coefficients or clipping speed masks the defect and corrupts every downstream interpretation.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

### 2.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 2.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 2.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 3 Case 2: Axis swap

### 3.1 Incident report

**Primary symptom:** A [batch,time,feature] tensor is passed as [time,batch,feature].

**Secondary evidence:** Training runs because dimensions are numerically compatible, but hidden state mixes examples and sequence length changes with batch size.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 3.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 3.3 High-information first probe

Use unequal dimensions such as batch=3, time=7, features=5; print named shape at model entry.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

### 3.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 3.5 Root cause

The dataset/collate output and model configuration disagree about batch_first.

## 3.6 Correct repair

Standardize one axis convention, assert it at the boundary, and transpose only in one named adapter when required.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 3.7 Regression test

A batch processed together must match each example processed separately with zero initial state.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 3.8 The tempting wrong repair

Increasing hidden size or training longer cannot repair wrong semantic axes.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 3.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 3.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

## 3.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 4 Case 3: Target leakage

## 4.1 Incident report

**Primary symptom:** Future speed accidentally appears in model inputs.

**Secondary evidence:** Validation error collapses unrealistically and remains strong even when controls are shuffled.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

## 4.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 4.3 High-information first probe

Print feature names and source frame indices for one window; train a linear model and inspect suspicious coefficient on future speed.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 4.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 4.5 Root cause

Window feature allow-list includes a target column at the same or later timestamp.

## 4.6 Correct repair

Separate state, future-control and target schemas; construct tensors from explicit allow-lists.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 4.7 Regression test

Assert no target feature/timestamp appears in history or future-input tensors and run a shuffled-target sanity test.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

### 4.8 The tempting wrong repair

Celebrating the score or adding regularization preserves the invalid experiment.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

### 4.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 4.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 4.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 5 Case 4: Random window split

### 5.1 Incident report

**Primary symptom:** Overlapping windows from one session enter every split.

**Secondary evidence:** Test performance is extremely stable and much better than unseen-session performance.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 5.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 5.3 High-information first probe

Report unique session IDs and raw frame IDs in each split; calculate overlap and nearest temporal distance.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 5.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 5.5 Root cause

Windows are generated first and then randomly assigned.

## 5.6 Correct repair

Assign sessions/events before windowing; persist deterministic split mapping.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 5.7 Regression test

No session ID or raw frame identifier may appear in more than one split.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 5.8 The tempting wrong repair

Reducing window overlap does not solve the generalization-claim mismatch.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 5.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 5.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 5.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 6 Case 5: Stale as-of join

### 6.1 Incident report

**Primary symptom:** Weather is forward-filled across a long outage.

**Secondary evidence:** Rain appears constant for minutes despite missing source records, and the model becomes overconfident.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 6.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 6.3 High-information first probe

Store matched source timestamp and compute age for every joined row; plot age distribution.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

### 6.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

### 6.5 Root cause

Forward fill or backward join has no tolerance and missingness is erased.

### 6.6 Correct repair

Use causal join with domain tolerance, carry age/missing masks, quarantine long gaps.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

### 6.7 Regression test

Every matched context timestamp must be ≤ frame time and age ≤ configured tolerance.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

### 6.8 The tempting wrong repair

Blind interpolation creates fabricated certainty rather than recovering evidence.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

### 6.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 6.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 6.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 7 Case 6: Normalizer leak

### 7.1 Incident report

**Primary symptom:** Standardization is fitted on all data.

**Secondary evidence:** Held-out features have suspiciously centred distributions and deployment transformation cannot be recreated from training artifacts.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 7.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 7.3 High-information first probe

Recompute normalizer hash from training rows only and compare; inspect fitted row identifiers.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 7.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 7.5 Root cause

Preprocessing is applied before split or fit is called on concatenated partitions.

## 7.6 Correct repair

Split first, fit on train, serialize statistics, transform validation/test without refit.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 7.7 Regression test

Changing validation/test values must not change stored normalizer statistics.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 7.8 The tempting wrong repair

The score inflation may be small, but the protocol and reproducibility are invalid.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 7.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 7.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 7.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 8 Case 7: Hidden-state reuse

### 8.1 Incident report

**Primary symptom:** Hidden state leaks between unrelated sessions.

**Secondary evidence:** Predictions depend on dataloader order and the first frames of a session contain information from a previous driver.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 8.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 8.3 High-information first probe

Run the same sessions in reversed batch order and compare outputs; print hidden norm at boundaries.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

### 8.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

### 8.5 Root cause

Stateful recurrence is used without entity/session reset policy.

## 8.6 Correct repair

Zero/reset hidden state at independent boundaries; make streaming state keyed and explicit.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 8.7 Regression test

Separate processing and batched processing with zero state must agree.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 8.8 The tempting wrong repair

Randomizing order can hide the symptom without defining correct state lifetime.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 8.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 8.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

## 8.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 9 Case 8: Missing eval mode

## 9.1 Incident report

**Primary symptom:** Dropout remains active during evaluation.

**Secondary evidence:** Repeated inference for identical input produces different predictions and reported metrics vary.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

## 9.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 9.3 High-information first probe

Call the model twice with fixed input/seed; inspect model.training.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 9.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 9.5 Root cause

Evaluation uses no_grad but never calls model.eval().

## 9.6 Correct repair

Switch to eval before validation/inference and restore train mode when resuming optimization.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 9.7 Regression test

Two deterministic eval calls must match for deterministic models; validation helper must assert mode.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 9.8 The tempting wrong repair

Averaging many stochastic passes is not a repair unless MC dropout is an intentional uncertainty method.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 9.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 9.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 9.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 10 Case 9: Gradient accumulation

### 10.1 Incident report

**Primary symptom:** Gradients are never cleared.

**Secondary evidence:** Update magnitude grows across batches, loss oscillates and gradient norms depend on batch position.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 10.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 10.3 High-information first probe

Print one parameter gradient before backward each iteration.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

### 10.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

### 10.5 Root cause

zero_grad is omitted while optimizer steps every batch.

### 10.6 Correct repair

Clear gradients at the intended accumulation boundary and document accumulation factor.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

### 10.7 Regression test

Two identical independent batches should produce identical gradients when starting from same parameters.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

### 10.8 The tempting wrong repair

Lowering learning rate may reduce symptoms but leaves unintended history in every update.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

### 10.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 10.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 10.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 11 Case 10: Exploding SSM

### 11.1 Incident report

**Primary symptom:** Unconstrained recurrent decay exceeds one.

**Secondary evidence:** Hidden norm grows exponentially during zero-input rollout, then outputs become inf/NaN.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

## 11.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 11.3 High-information first probe

Plot hidden norm over 200 zero-input steps and inspect transition eigenvalues/gates.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 11.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 11.5 Root cause

State propagation permits amplification greater than one without stabilizing design.

## 11.6 Correct repair

Use stable parameterization/bounds, initialization and horizon stress tests; clip only as secondary protection.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 11.7 Regression test

Bounded input must produce finite bounded hidden state over the certified horizon.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 11.8 The tempting wrong repair

Output clipping conceals unstable internal memory and gradients.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

### 11.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 11.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 11.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 12 Case 11: KL collapse

### 12.1 Incident report

**Primary symptom:** RSSM posterior ignores stochastic state.

**Secondary evidence:** KL is nearly zero, posterior and prior are identical, and decoder predictions barely change when latent is resampled.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 12.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 12.3 High-information first probe

Report KL per latent dimension, latent variance and decoder sensitivity to z.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

### 12.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 12.5 Root cause

KL pressure/decoder capacity lets deterministic path solve loss while stochastic latent carries no information.

## 12.6 Correct repair

Tune balance with warm-up/free-nats/capacity changes after diagnostics; force evaluation of prior and latent probes.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 12.7 Regression test

Changing z while holding h fixed should alter decoded predictions on a task designed to require uncertainty.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 12.8 The tempting wrong repair

Simply increasing stochastic dimension adds unused variables and can worsen optimization.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 12.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 12.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

## 12.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 13 Case 12: Planner exploitation

## 13.1 Incident report

**Primary symptom:** CEM finds actions outside training support.

**Secondary evidence:** Imagined reward is excellent but actions saturate at boundaries and fail in trusted simulator/replay.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

## 13.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 13.3 High-information first probe

Measure action-distribution distance, uncertainty and violations for elite candidates.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 13.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 13.5 Root cause

Reward/model lacks constraints and planner searches OOD regions where dynamics are inaccurate.

## 13.6 Correct repair

Enforce bounds, OOD/uncertainty penalties, trusted-model validation and receding-horizon replanning.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 13.7 Regression test

A hidden loophole fixture must be rejected or penalized despite high predicted reward.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

### 13.8 The tempting wrong repair

Reducing CEM iterations may hide exploitation but does not repair model/reward validity.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

### 13.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

### 13.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 13.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

## 14 Case 13: Artifact overwrite

### 14.1 Incident report

**Primary symptom:** Two runs write the same model path.

**Secondary evidence:** Metrics refer to one config while checkpoint silently belongs to another; rollback is impossible.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

### 14.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

### 14.3 High-information first probe

Compare checkpoint hash, config hash and timestamps in registry/run directory.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 14.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 14.5 Root cause

Mutable latest/model.pt is used as both identity and storage.

## 14.6 Correct repair

Use immutable run IDs/content hashes; make latest a pointer, not the sole artifact.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 14.7 Regression test

Concurrent runs must produce distinct immutable paths and provenance must resolve exactly.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 14.8 The tempting wrong repair

Adding timestamps to filenames without registry metadata still weakens lineage.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 14.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 14.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

### 14.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 15 Case 14: Airflow payload

## 15.1 Incident report

**Primary symptom:** Large arrays are passed through XCom.

**Secondary evidence:** Scheduler metadata grows, serialization fails and workers spend time moving data through the control plane.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

## 15.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 15.3 High-information first probe

Inspect XCom sizes and task logs; trace where durable files already exist.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 15.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 15.5 Root cause

Task interfaces return in-memory arrays instead of artifact references.

## 15.6 Correct repair

Persist arrays in object/file storage and pass small path/ID/hash messages.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 15.7 Regression test

Task output metadata must remain below a chosen size and downstream task must reload/validate artifact.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 15.8 The tempting wrong repair

Raising database limits turns an architectural problem into a larger operational problem.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 15.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 15.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

## 15.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

# 16 Case 15: UI false precision

## 16.1 Incident report

**Primary symptom:** The UI reports more precision than the model supports.

**Secondary evidence:** A scenario displays speed to 0.001 km/h beyond the measured error and shows untrusted horizons as equally certain.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

## 16.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

## 16.3 High-information first probe

Compare displayed digits/horizon with calibration and held-out metrics; reproduce one point outside UI.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

## 16.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

## 16.5 Root cause

Presentation is designed independently of model uncertainty, provenance and trusted range.

## 16.6 Correct repair

Round to meaningful resolution, label trusted horizon, show run/model identity and exploratory regions.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

## 16.7 Regression test

Snapshot/UI test verifies provenance, units, rounding and horizon labels for a known run.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

## 16.8 The tempting wrong repair

A beautiful chart can increase harm when it makes uncertain outputs look authoritative.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

## 16.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

## 16.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

## 16.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.